

Randy's code for Week 4 Lecture

Table of contents

1	The function from Breukelen's paper, based on numerical solution	3
2	The solution from lecture slides, calculation for the cubic equaiton	5
3	The results	7
3.1	Checking for K	7
3.2	Checking for Rho	8
3.3	Checking for CV_m	9

Below is Randy's code for the comparison between two methods for $\bar{m}$. `m_sol` denotes the numerical solution obtained using `uniroot()` to numerically solve the result. `m_cal` denotes the calculated solution obtained from the cubic formula presented in the lecture slides.

$$CV = \frac{SD}{\bar{m}}$$

$$K = \frac{(Z_{\alpha/2} + Z_{\beta})^2 (2\sigma^2) [1 + (\bar{m} - 1)\rho] RE(\bar{m})}{(\mu_1 - \mu_2)^2}$$

$$RE(\bar{m}) = \left(1 - CV_m^2 \frac{\rho \bar{m}}{\rho \bar{m} + 1 - \rho} \left[1 - \frac{\rho \bar{m}}{\rho \bar{m} + 1 - \rho} \right] \right)^{-1}$$

1 The function from Breukelen's paper, based on numerical solution

```
K_m <- function(m_bar,
               alpha = 0.05,
               power = 0.8,
               mu1 = 5,
               mu2 = 6,
               sigma = 1,
               rho = 0.05,
               cv_m = 0.50) {

  ## the normal z quantiles
  z_alpha <- qnorm(1 - alpha / 2)
  z_beta  <- qnorm(power)

  ## the design effect
  de <- 1 + (m_bar - 1) * rho

  ## the fraction appearing in RE(m_bar)
  f_re <- rho * m_bar / (rho * m_bar + 1 - rho)

  # calculate RE as a function of m_bar
  re_dn <- 1 - cv_m^2 * f_re * (1 - f_re)

  if (any(re_dn <= 0)) {
    stop("These parameter value RE > 0.")
  }

  re <- 1 / re_dn

  ## calculate K(m_bar); sigma2 is the variance, sigma^2
  K <- (z_alpha + z_beta)^2 * (2 * sigma ^ 2) * de * re /
    (m_bar * (mu1 - mu2)^2)
```

```
    return(K)
}
```

```
solve_m <- function(K_target,
                    interval,
                    ...) {

  ## solve  $K(m\_bar) - K\_target = 0$ 
  result <- uniroot(f = function(m_bar) K_m(m_bar, ...) - K_target,
                   interval = interval)

  return(result$root)
}
```

```
m_s0 <- solve_m(K_target = 6,
               interval = c(1, 1000),
               mu1 = 5,
               mu2 = 6,
               sigma = 1,
               rho = 0.05,
               cv_m = 0.50,
               alpha = 0.05,
               power = 0.8)
```

```
m_s0
```

```
#> [1] 2.95878
```

2 The solution from lecture slides, calculation for the cubic equation

```
alpha = 0.05
power = 0.80
mu1 = 5
mu2 = 6
sigma = 1
rho = 0.05
cv_m = 0.50
K_target = 6

z_alpha <- qnorm(1 - alpha / 2)
z_beta <- qnorm(power)
L = K_target * (mu2 - mu1)^2 / ((z_alpha + z_beta)^2 * 2 * sigma^2)

a = (L - rho) * rho ^2
b = 2 * rho * (1 - rho) * (L - rho) - (1 - rho) * rho^2 - L * cv_m^2 * rho * (1 - rho)
c = (1 - rho)^2 * (L - 3 * rho)
d = -(1 - rho)^3

p <- (3 * a * c - b^2) / (3 * a^2)
q <- (27 * a^2 * d - 9 * a * b * c + 2 * b^3) / (27 * a^3)
delta <- (q / 2)^2 + (p / 3)^3

m_bar = -b / (3 * a) + (-q / 2 + sqrt(delta))^(1/3) +
        (-q / 2 - sqrt(delta))^(1/3)

m_bar
```

```
#> [1] 2.958767
```

```
calculate_m <- function(K_target = 6,
                        alpha = 0.05,
```

```

        power = 0.8,
        mu1 = 5,
        mu2 = 6,
        sigma = 1,
        rho = 0.05,
        cv_m = 0.50){
  z_alpha <- qnorm(1 - alpha / 2)
  z_beta <- qnorm(power)
  L = K_target * (mu2 - mu1)^2 / ((z_alpha + z_beta)^2 * 2 * sigma^2)
  a = (L - rho) * rho ^2
  b = 2 * rho * (1 - rho) * (L - rho) - (1 - rho) * rho^2 - L * cv_m^2 * rho * (1 - rho)
  c = (1 - rho)^2 * (L - 3 * rho)
  d = -(1 - rho)^3

  p <- (3 * a * c - b^2) / (3 * a^2)
  q <- (27 * a^2 * d - 9 * a * b * c + 2 * b^3) / (27 * a^3)
  delta <- (q / 2)^2 + (p / 3)^3

  m_bar = -b / (3 * a) + (-q / 2 + sqrt(delta))^(1/3) +
    (-q / 2 - sqrt(delta))^(1/3)

  return(m_bar)
}

## here for the calculation
m_c0 <- calculate_m()
m_c0

```

```
#> [1] 2.958767
```

```

## the difference between the numeric
## calculated and the solved equation
m_c0 - m_s0

```

```
#> [1] -1.355366e-05
```

3 The results

The equation for the lecture slides is doing just as well as the numeric solution for most of the cases (when we can calculate the equation). There are cases we cannot solve the cubic equations I guess when the RE is not positive. Here are some of the results.

The numerical and cubic approaches provide closely agreeing estimates of the mean cluster size. But the cubic equation sometimes is not solvable given certain setups.

3.1 Checking for K

```
library(purrr)
## K_target; keep rho = 0.05 and cv_m = 0.50
K_values <- 2:11

Km_cal <- data.frame(K_target = K_values,
                     m_bar = map_dbl(K_values,
                                     ~ calculate_m(K_target = .x,
                                                    mu1 = 5,
                                                    mu2 = 6,
                                                    sigma = 1,
                                                    rho = 0.05,
                                                    cv_m = 0.50,
                                                    alpha = 0.05,
                                                    power = 0.8)))

Km_sol <- data.frame(K_target = K_values,
                     m_bar = map_dbl(K_values,
                                     ~ solve_m(K_target = .x,
                                                interval = c(1, 1000),
                                                mu1 = 5,
                                                mu2 = 6,
                                                sigma = 1,
                                                rho = 0.05,
                                                cv_m = 0.50,
```

```

alpha = 0.05,
power = 0.8)))

## Error message: Error in `map_dbl()`:
##   In index: 12.
## Caused by error in `uniroot()`:
##   ! f() values at end points not of opposite sign
## Run `rlang::last_trace()` to see where the error occurred.

bind_cols(Km_cal, m_sol = Km_sol$m_bar, diff = abs(Km_cal$m_bar - Km_sol$m_bar))

#>   K_target    m_bar    m_sol      diff
#> 1      2 13.637918 13.637913 5.384382e-06
#> 2      3  7.220130  7.220130 1.754336e-07
#> 3      4  4.885610  4.885619 9.409925e-06
#> 4      5  3.686752  3.686744 8.320899e-06
#> 5      6  2.958767  2.958780 1.355366e-05
#> 6      7  2.470274  2.470274 4.363650e-07
#> 7      8  2.119959  2.119957 1.933748e-06
#> 8      9  1.856530  1.856525 5.363702e-06
#> 9     10  1.651262  1.651262 1.460023e-07
#> 10     11  1.486824  1.486824 1.413418e-07

```

3.2 Checking for Rho

```

rho_values <- seq(0.01, 0.50, by = 0.05)

Rm_cal <- data.frame(rho = rho_values,
                    m_bar = map_dbl(rho_values,
                                   ~ calculate_m(rho = .x,
                                                  mu1 = 5,
                                                  mu2 = 6,
                                                  sigma = 1,
                                                  K_target = 10,
                                                  cv_m = 0.50,
                                                  alpha = 0.05,
                                                  power = 0.8)))

Rm_sol <- data.frame(rho = rho_values,

```

```

        m_bar = map_dbl(rho_values,
                        ~ solve_m(K_target = 10,
                                interval = c(1, 1000),
                                mu1 = 5,
                                mu2 = 6,
                                sigma = 1,
                                rho = .x,
                                cv_m = 0.50,
                                alpha = 0.05,
                                power = 0.8)))

bind_cols(Rm_cal, m_sol = Rm_sol$m_bar, diff = abs(Rm_cal$m_bar - Rm_sol$m_bar))

```

```

#>      rho    m_bar    m_sol      diff
#> 1  0.01 1.585107 1.585105 2.557538e-06
#> 2  0.06 1.669104 1.669104 1.016096e-08
#> 3  0.11 1.767270 1.767270 4.803438e-09
#> 4  0.16 1.883028 1.883026 1.883913e-06
#> 5  0.21 2.021363 2.021361 2.782511e-06
#> 6  0.26 2.190031 2.190021 9.789879e-06
#> 7  0.31 2.401866 2.401872 5.335659e-06
#> 8  0.36 2.679522 2.679523 7.188421e-07
#> 9  0.41 3.066318 3.066312 5.882169e-06
#> 10 0.46 3.655297 3.655325 2.753415e-05

```

3.3 Checking for CV_m

```

cv_values <- seq(0.10, 2, by = 0.1)

CVm_cal <- data.frame(cv = cv_values,
                     m_bar = map_dbl(cv_values,
                                     ~ calculate_m(cv_m = .x,
                                                     mu1 = 5,
                                                     mu2 = 6,
                                                     sigma = 1,
                                                     K_target = 6,
                                                     rho = 0.05,
                                                     alpha = 0.05,
                                                     power = 0.8)))

```

```
CVm_sol <- data.frame(cv = cv_values,
                     m_bar = map_dbl(cv_values,
                                     ~ solve_m(cv_m = .x,
                                                interval = c(1, 1000),
                                                mu1 = 5,
                                                mu2 = 6,
                                                sigma = 1,
                                                rho = 0.05,
                                                K_target = 6,
                                                alpha = 0.05,
                                                power = 0.8)))
```

```
bind_cols(Rm_cal, m_sol = Rm_sol$m_bar, diff = abs(Rm_cal$m_bar - Rm_sol$m_bar))
```

```
#>      rho    m_bar    m_sol      diff
#> 1  0.01 1.585107 1.585105 2.557538e-06
#> 2  0.06 1.669104 1.669104 1.016096e-08
#> 3  0.11 1.767270 1.767270 4.803438e-09
#> 4  0.16 1.883028 1.883026 1.883913e-06
#> 5  0.21 2.021363 2.021361 2.782511e-06
#> 6  0.26 2.190031 2.190021 9.789879e-06
#> 7  0.31 2.401866 2.401872 5.335659e-06
#> 8  0.36 2.679522 2.679523 7.188421e-07
#> 9  0.41 3.066318 3.066312 5.882169e-06
#> 10 0.46 3.655297 3.655325 2.753415e-05
```